



Автономная некоммерческая профессиональная образовательная организация  
«МЕЖДУНАРОДНЫЙ ВОСТОЧНО-ЕВРОПЕЙСКИЙ КОЛЛЕДЖ»  
Пушкинская ул., д. 268, 426008, г. Ижевск. Тел.: (3412) 77-68-24. E-mail: mveu@mveu.ru, www.mveu.ru  
ИНН 1831200089. ОГРН 1201800020641

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ  
ФЕДЕРАЦИИ  
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ  
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ  
«БАЛТИЙСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ ИМЕНИ ИММАНУИЛА  
КАНТА»**

**ОТЧЕТ**

по учебно-технологической практике  
на тему: «Разработка сетевого модуля игровой платформы Vobux»  
студента 1 курса группы [номер группы]  
специальности 01.03.02 Прикладная математика и информатика  
[ФИО студента]

Отметка о защите отчета

Отчет защищен с оценкой \_\_\_\_\_

«\_\_» \_\_\_\_\_ 20\_\_ г.

Руководитель учебно-технологической практики \_\_\_\_\_ [ФИО  
руководителя]

Калининград 2026



Автономная некоммерческая профессиональная образовательная организация

«МЕЖДУНАРОДНЫЙ ВОСТОЧНО-ЕВРОПЕЙСКИЙ КОЛЛЕДЖ»

Пушкинская ул., д. 268, 426008, г. Ижевск. Тел.: (3412) 77-68-24. E-mail: mveu@mveu.ru, www.mveu.ru  
ИНН 1831200089. ОГРН 1201800020641

## Содержание

|                                                            |    |
|------------------------------------------------------------|----|
| Введение                                                   | 3  |
| 1. Описание постановки задачи                              | 5  |
| 2. Структура ролей пользователей                           | 7  |
| 3. Обоснование выбора технологий                           | 9  |
| 4. Описание архитектуры сетевого модуля                    | 11 |
| 5. Описание программных модулей регистрации и авторизации  | 13 |
| 6. Описание программного модуля пользовательского контента | 14 |
| 7. Описание программного модуля игровых серверов           | 16 |
| 8. Описание административного модуля статистики            | 17 |
| 9. Тестирование сетевого модуля                            | 18 |
| Заключение                                                 | 19 |
| Список литературы                                          | 20 |
| Приложение А. Основные файлы проекта                       | 21 |
| Приложение Б. Сценарий демонстрации                        | 22 |



## Введение

Учебно-технологическая практика была связана с проектированием, реализацией и проверкой сетевого модуля игровой платформы Vobux. Платформа разрабатывается как многопользовательская среда, в которой игроки могут регистрироваться, входить в учетную запись, создавать карты, публиковать игровые предметы, взаимодействовать с друзьями и подключаться к игровым серверам. Центральной частью такой системы является сетевой модуль, обеспечивающий связь между клиентом, сервером приложений, базой данных и игровыми комнатами.

Актуальность выбранной темы определяется тем, что современные игровые платформы фактически являются распределенными информационными системами. Даже если визуальная часть создается в игровом движке, основные данные должны храниться на сервере: учетные записи, профили игроков, список друзей, каталог предметов, инвентарь, карты, статистика посещений и состояние активных серверов. Без устойчивого сетевого слоя такая платформа не может масштабироваться и не может считаться полноценным сервисом.

Целью работы является разработка и документирование сетевого модуля игровой платформы Vobux, который обеспечивает регистрацию и авторизацию пользователей, хранение профилей и пользовательского контента, загрузку данных каталога, работу с социальными связями, обмен сведениями об активных игровых серверах и предоставление административной статистики.

Для достижения цели были поставлены следующие задачи:

изучить требования учебно-технологической практики к веб-сервису с базой данных, регистрацией, авторизацией, загрузкой изображений, поиском и разграничением прав;

определить роли пользователей и перечень данных, которые должны храниться на стороне сервера;

реализовать взаимодействие клиента Godot с серверным API и базой PocketBase;

обеспечить хранение карт, предметов, файлов и статистики в едином серверном контуре;

подготовить административную статистику, позволяющую оценивать фактическое состояние проекта;



проверить работоспособность сетевого модуля на реальных данных и оформить результаты в виде отчета.

Объектом исследования является игровая платформа Vobux как программная система. Предметом исследования является сетевой модуль этой платформы, включающий клиент-серверное взаимодействие, базу данных, файловое хранилище, механизм авторизации, систему пользовательского контента и статистический административный интерфейс.

В ходе практики использовались Godot 4, GDScript, Node.js, Express, PocketBase, HTTP API, WebSocket-соединения, файловое хранилище на VPS, а также инструменты проверки серверного состояния. Итогом работы стал модуль, который можно использовать как основу для дальнейшего расширения игровой платформы.



## **1. Описание постановки задачи**

Техническое задание на разработку сетевого модуля игровой платформы Bobux

### **1.1. Назначение документа**

Настоящее техническое задание определяет требования к сетевому модулю игровой платформы Bobux. Документ описывает назначение системы, роли пользователей, функциональные возможности, состав программных модулей, требования к безопасности, хранению данных и проверке результата.

### **1.2. Цель проекта**

Цель проекта состоит в создании серверной части игровой платформы, которая позволяет пользователю зарегистрироваться, пройти авторизацию, получить персональный профиль, создавать и публиковать карты, получать предметы из каталога, управлять аватаром, видеть друзей и подключаться к активным игровым серверам.

### **1.3. Общие требования**

Система должна работать по принципу «сервер является главным источником данных». Клиент Godot отображает интерфейс и отправляет запросы, но не считается доверенным источником для важных операций. Все критичные действия, включая вход, сохранение профиля, получение инвентаря, публикацию контента и получение статистики, проходят через серверный API.

Пользователь без авторизации должен иметь возможность открыть клиент, увидеть форму входа и зарегистрировать учетную запись. Авторизованный пользователь получает доступ к профилю, списку карт, каталогу предметов, друзьям, инвентарю, аватару и игровым сессиям. Администратор должен иметь возможность получать статистику по базе данных и состоянию сервисов.

### **1.4. Функциональные требования**

Сетевой модуль должен поддерживать регистрацию, вход, загрузку профиля, сохранение данных аватара, публикацию карт, хранение превью и файлов, получение списка карт, работу с друзьями, получение каталога предметов, выдачу предметов пользователю, хранение инвентаря, регистрацию активных игровых серверов и получение административной статистики.

Для соответствия заданию практики в проекте предусмотрены функции работы с базой данных, разграничение доступа для неавторизованного и авторизованного



пользователя, загрузка изображений и файлов, создание, редактирование и удаление записей, поиск и вывод информации, а также серверная проверка входящих данных.

### **1.5. Нефункциональные требования**

Система должна быть устойчивой к некорректным данным клиента, поддерживать расширение количества коллекций в базе, не блокировать вход пользователя при частичной недоступности второстепенных данных, а также предоставлять понятные ошибки для клиента. При загрузке файлов должны учитываться ограничения размера и допустимые форматы.

Серверный модуль должен быть развернут на VPS, запускаться как отдельный процесс, перезапускаться средствами process manager, работать за HTTP-прокси и иметь отдельные endpoints для проверки состояния. Данные должны храниться централизованно в RocketBase и файловом хранилище, что позволяет переносить клиентские сборки без потери пользовательского состояния.



## 2. Структура ролей пользователей

В сетевом модуле выделены три основные роли: неавторизованный посетитель, авторизованный игрок и администратор. Такое деление позволяет разделить права доступа и не раскрывать серверные функции пользователю, который не прошел вход.

Неавторизованный посетитель может открыть клиент и выполнить регистрацию или вход. На этом этапе пользователь не получает доступ к персональным данным, инвентарю, списку друзей и операциям публикации контента. Такой подход снижает риск несанкционированного изменения данных и соответствует требованию разграничения прав.

Авторизованный игрок является основной ролью системы. Он может загружать свой профиль, просматривать карты, получать и надевать предметы, сохранять настройки аватара, работать с друзьями, создавать карты и подключаться к игровым серверам. При этом клиент не изменяет данные напрямую в базе: все операции проходят через API.

Администратор получает доступ к статистике и служебной информации. В рамках практической работы был расширен административный endpoint, который позволяет получить количество учетных записей, профилей, карт, предметов, записей инвентаря, дружеских связей, размер базы PocketBase и размер файлового хранилища.

Отдельно можно выделить служебную роль игрового сервера. Игровой сервер сообщает API о своем состоянии, количестве игроков и доступности комнаты. Для этого используется heartbeat-механизм, который не должен быть доступен обычным пользователям.

| Роль                          | Доступные функции                                                                  |
|-------------------------------|------------------------------------------------------------------------------------|
| Неавторизованный пользователь | Регистрация, вход, получение публичной информации                                  |
| Авторизованный игрок          | Профиль, друзья, карты, каталог, инвентарь, аватар, подключение к игровым серверам |
| Администратор                 | Статистика базы данных, контроль сервисов, анализ состояния платформы              |
| Игровой сервер                | Регистрация активной комнаты, heartbeat, передача сведений о сессии                |

Сведения, приведенные в таблице, используются в сетевом модуле как контрольные точки: по ним можно проверить, какие данные хранятся на сервере, какие операции выполняет клиент и какие результаты должны быть получены при демонстрации проекта.



Автономная некоммерческая профессиональная образовательная организация

**«МЕЖДУНАРОДНЫЙ ВОСТОЧНО-ЕВРОПЕЙСКИЙ КОЛЛЕДЖ»**

---

Пушкинская ул., д. 268, 426008, г. Ижевск. Тел.: (3412) 77-68-24. E-mail: mveu@mveu.ru, www.mveu.ru  
ИНН 1831200089. ОГРН 1201800020641

Такое представление удобно для защиты, поскольку показывает не только текстовое описание, но и связь между требованиями практики, архитектурой проекта и фактической реализацией в исходном коде.



### 3. Обоснование выбора технологий

Для клиентской части используется игровой движок Godot 4. Он подходит для проекта, так как позволяет создавать 3D-игру, пользовательский интерфейс, редактор карт и сетевое взаимодействие в единой среде. Клиентская логика написана на GDScript, что упрощает интеграцию с узлами сцены, интерфейсом и игровыми объектами.

Серверный API реализован на Node.js с использованием Express. Выбор этой технологии обусловлен удобной работой с HTTP-запросами, большим количеством библиотек, простотой развертывания на VPS и хорошей совместимостью с JSON-форматом, который используется клиентом Godot.

Для хранения данных используется PocketBase. Это компактная серверная база данных, объединяющая SQLite, REST API, коллекции, пользователей и файловое хранилище. Для учебного проекта PocketBase удобен тем, что позволяет быстро описывать коллекции, хранить записи и связывать их с файлами.

Для обмена данными между клиентом и сервером используются HTTP-запросы. Такой подход подходит для операций входа, загрузки профиля, получения каталога, сохранения аватара, публикации карты и запроса статистики. Для игровых сессий используется отдельный сетевой слой Godot, потому что передача игрового состояния требует более частого обмена событиями.

Для эксплуатации на сервере применяются VPS, Nginx и PM2. Nginx выполняет маршрутизацию публичных запросов, а PM2 поддерживает Node.js API в рабочем состоянии и перезапускает процесс при сбоях. Такой набор технологий соответствует реальному сценарию развертывания веб-сервиса.

Выбранная архитектура оставляет возможность дальнейшего расширения: можно добавлять новые коллекции, новые API-маршруты, отдельные игровые серверы, дополнительные административные отчеты и более сложную систему прав доступа. Это важно, поскольку игровая платформа должна развиваться постепенно.

| Компонент   | Используемая технология | Назначение                                              |
|-------------|-------------------------|---------------------------------------------------------|
| Клиент      | Godot 4, GDScript       | Интерфейс игры, лобби, редактор, подключение к серверам |
| API         | Node.js, Express        | Маршруты авторизации, профилей, каталога и статистики   |
| База данных | PocketBase              | Хранение пользователей, карт,                           |



|                |                   |                                                      |
|----------------|-------------------|------------------------------------------------------|
|                |                   | предметов и социальных связей                        |
| Сетевые сессии | Godot multiplayer | Игровые комнаты и обмен событиями между игроками     |
| Развертывание  | VPS, Nginx, PM2   | Публичный доступ, проксирование и контроль процессов |

Сведения, приведенные в таблице, используются в сетевом модуле как контрольные точки: по ним можно проверить, какие данные хранятся на сервере, какие операции выполняет клиент и какие результаты должны быть получены при демонстрации проекта.

Такое представление удобно для защиты, поскольку показывает не только текстовое описание, но и связь между требованиями практики, архитектурой проекта и фактической реализацией в исходном коде.



#### 4. Описание архитектуры сетевого модуля

Сетевой модуль Bobux построен по многоуровневой схеме. На первом уровне находится клиент Godot, который отвечает за интерфейс, сбор действий пользователя и отображение данных. На втором уровне расположен серверный API, принимающий запросы и выполняющий проверку данных. На третьем уровне находится база PocketBase и файловое хранилище.

Клиентская часть обращается к серверу через модуль `cloud_api.gd`. В этом модуле сосредоточены функции входа, регистрации, загрузки профиля, получения друзей, карт, каталога и сохранения аватара. Такое разделение удобно, потому что интерфейс не должен знать детали HTTP-запросов и формат конкретных серверных маршрутов.

Серверная часть реализована в файле `services/bobux_api/server.js`. Она принимает запросы клиента, проверяет параметры, обращается к PocketBase и возвращает унифицированные JSON-ответы. Серверный слой также содержит обработку ошибок, нормализацию данных и административные функции.

PocketBase используется как основное хранилище. В нем находятся коллекции профилей, карт, активных серверов, моделей, предметов аватара, каталога, инвентаря и дружеских связей. Файлы, связанные с картами и предметами, хранятся отдельно, но доступны через публичные URL.

Игровой сервер является отдельным процессом Godot. Он отвечает за конкретную игровую комнату и взаимодействие игроков внутри нее. API хранит сведения об активных серверах, что позволяет лобби показывать доступные комнаты и подключать игрока к нужному адресу.

Архитектура построена так, чтобы клиент не мог самостоятельно назначить себе предмет, подделать профиль или изменить статистику карты напрямую. Все такие действия должны проходить через сервер, где можно проверить пользователя и корректность операции.



### Архитектура сетевого модуля Vobux

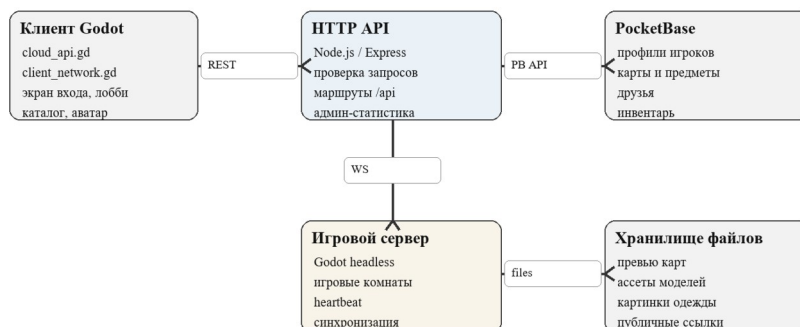


Рисунок 1 - Архитектура сетевого модуля игровой платформы Vobux



## 5. Описание программных модулей регистрации и авторизации

Модуль регистрации и авторизации является входной точкой сетевой системы. Он решает две задачи: подтверждает личность пользователя и связывает игрового клиента с серверным профилем. Без этого этапа невозможно корректно хранить инвентарь, друзей, созданные карты и настройки аватара.

При регистрации клиент передает имя пользователя, адрес электронной почты или иной идентификатор и пароль. Сервер проверяет допустимость данных, создает учетную запись в PocketBase и формирует связанный профиль игрока. Профиль нужен для публичных данных, которые отображаются другим пользователям.

При входе пользователь отправляет учетные данные на API. Сервер обращается к PocketBase, получает auth-запись и возвращает клиенту результат авторизации. После успешного входа клиент последовательно загружает профиль, данные аватара, друзей, инвентарь, каталог и список карт.

Важной особенностью реализации является разделение auth-записи и игрового профиля. Auth-запись отвечает за безопасность входа, а профиль хранит игровые данные: имя, статус, внешний вид, статистику и связи с пользовательским контентом. Такое разделение упрощает дальнейшее развитие проекта.

Для повышения устойчивости входа сетевой модуль должен обрабатывать частичные ошибки. Например, если список друзей временно не загрузился, клиент не должен полностью зависать после авторизации. Основной профиль и токен входа являются критичными, а вторичные данные могут догружаться отдельно.

В рамках подготовки проекта к защите был описан и проверен сценарий загрузки профиля после входа. Он показывает, какие компоненты участвуют в операции и почему сервер остается главным источником данных.



Автономная некоммерческая профессиональная образовательная организация

«МЕЖДУНАРОДНЫЙ ВОСТОЧНО-ЕВРОПЕЙСКИЙ КОЛЛЕДЖ»

Пушкинская ул., д. 268, 426008, г. Ижевск. Тел.: (3412) 77-68-24. E-mail: mveu@mveu.ru, www.mveu.ru  
ИНН 1831200089. ОГРН 1201800020641

### Сценарий авторизации и загрузки профиля

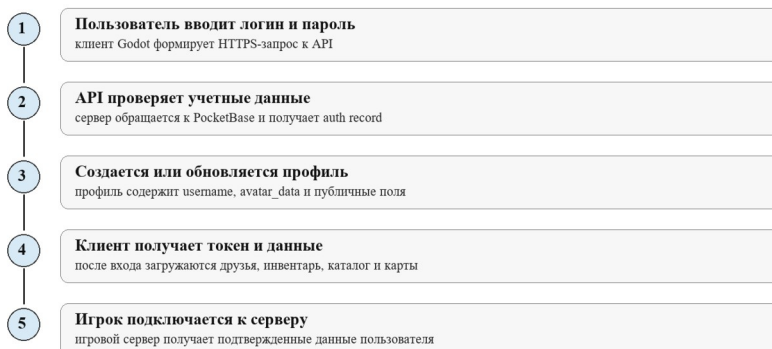


Рисунок 2 - Сценарий авторизации и загрузки профиля



## 6. Описание программного модуля пользовательского контента

Пользовательский контент является важной частью игровой платформы. В Vobux к нему относятся карты, модели, предметы аватара, превью, изображения одежды, записи инвентаря и сведения о публикации. Сетевой модуль отвечает за то, чтобы эти данные сохранялись централизованно и могли быть загружены с любого клиента.

Модуль карт позволяет игроку создать карту в редакторе, сохранить ее на сервере и отобразить в списке доступных режимов. Для каждой карты хранятся название, владелец, статус публикации, счетчики посещений и лайков, дата обновления и путь к превью. Это позволяет строить каталог карт и сортировать их по популярности.

Модуль предметов аватара хранит данные о вещах, которые создаются пользователями. Для каждого предмета сохраняется тип, название, автор, публичность, ссылка на превью и данные, необходимые для применения предмета к персонажу. Такой подход позволяет отделить описание предмета от самого клиента.

Инвентарь связывает пользователя и предмет. Если игрок получил предмет или создал его, в базе появляется запись, по которой клиент понимает, какие вещи доступны данному игроку. Это важно для защиты от ситуации, когда пользователь пытается надеть предмет, которого у него нет.

Файловое хранилище используется для превью, изображений, ассетов карт и иных данных, которые нецелесообразно хранить внутри JSON-записей. Ссылки на файлы сохраняются в коллекциях PocketBase, а клиент получает готовый URL для загрузки.

В задании практики отдельно указаны операции создания, редактирования, удаления и поиска записей. В контексте Vobux эти операции реализуются через карты, предметы, профиль, друзей и каталог. Пользователь создает карту или предмет, редактирует его параметры, может удалить или скрыть запись, а клиент выполняет поиск и фильтрацию по спискам.

| Коллекция      | Назначение                      | Количество записей |
|----------------|---------------------------------|--------------------|
| profiles       | Публичные профили игроков       | 654                |
| maps           | Пользовательские карты и режимы | 86                 |
| avatar_items   | Предметы аватара                | 83                 |
| user_inventory | Связь пользователя с предметами | 1077               |
| friendships    | Социальные связи игроков        | 172                |
| active_servers | Состояние игровых комнат        | 0                  |



Сведения, приведенные в таблице, используются в сетевом модуле как контрольные точки: по ним можно проверить, какие данные хранятся на сервере, какие операции выполняет клиент и какие результаты должны быть получены при демонстрации проекта.

Такое представление удобно для защиты, поскольку показывает не только текстовое описание, но и связь между требованиями практики, архитектурой проекта и фактической реализацией в исходном коде.



## 7. Описание программного модуля игровых серверов

Модуль игровых серверов отвечает за связь между лобби и конкретной игровой комнатой. Пользователь в лобби выбирает карту, после чего клиент должен получить адрес сервера, подключиться к нему и передать подтвержденные данные игрока. Если такой модуль отсутствует, платформа превращается только в локальный редактор без полноценного мультиплеера.

В Vobux серверная часть разделяет HTTP API и игровой процесс. API хранит данные и выдает сведения о доступных серверах, а игровой сервер занимается синхронизацией игроков внутри карты. Это снижает нагрузку на API и позволяет масштабировать игровые комнаты отдельно.

Игровой сервер периодически отправляет heartbeat. Такой запрос сообщает, что сервер жив, какая карта запущена, сколько игроков подключено и куда можно подключаться. Если heartbeat перестает приходить, запись сервера может считаться устаревшей и не должна показываться игрокам как доступная.

Клиентский модуль `client_network.gd` отвечает за подключение к игровому серверу. Он получает адрес, открывает сетевое соединение и синхронизирует игровые сущности. Для пользователя этот процесс выглядит как переход из лобби в выбранный режим.

Принцип «сервер главный» также важен в игровых сессиях. Клиент может отправлять действия, но окончательное состояние игрового мира должно подтверждаться сервером. Такой подход нужен для защиты от подмены позиции, инвентаря, состояния аватара и иных игровых данных.

В дальнейшем модуль игровых серверов может быть расширен системой отдельных комнат, очередями подключения, автоматическим запуском серверов под популярные карты и передачей статистики посещений после завершения игровой сессии.



## 8. Описание административного модуля статистики

Административный модуль статистики нужен для контроля состояния игровой платформы. Он позволяет понять, сколько пользователей зарегистрировано, сколько профилей создано, сколько карт и предметов находится в базе, сколько записей хранится в инвентаре и какой объем занимает файловое хранилище.

В рамках работы был расширен endpoint административной статистики. Теперь он возвращает не только количество основных сущностей, но и сведения о размере данных PocketBase, размере публичного хранилища, количестве записей в ключевых коллекциях и состоянии активных серверов.

Такая статистика полезна при защите проекта, потому что показывает, что система не является демонстрационной страницей без данных. В базе уже есть реальные записи пользователей, карт, предметов, инвентаря и социальных связей. Следовательно, сетевой модуль обслуживает живые данные платформы.

На момент проверки в базе содержалось 612 учетных записей авторизации, 654 профиля игроков, 86 карт, 83 предмета аватара, 1077 записей инвентаря и 172 дружеских связей.

Размер данных PocketBase составил около 2005.53 МБ, а размер публичного файлового хранилища составил около 411.25 МБ. Эти показатели подтверждают, что модуль работает не только с текстовыми записями, но и с файлами пользовательского контента.

Административная статистика может использоваться для дальнейшего развития проекта: определения популярных карт, анализа роста базы, поиска перегруженных коллекций, контроля активности игровых серверов и подготовки технических отчетов.



## 9. Тестирование сетевого модуля

Проверка сетевого модуля выполнялась на нескольких уровнях. На уровне исходного кода проверялся синтаксис серверного файла Node.js. На уровне проекта Godot выполнялась headless-проверка загрузки проекта. На уровне сервера проверялся публичный endpoint состояния API.

Функциональное тестирование включало сценарии входа пользователя, загрузки профиля, получения данных каталога, получения списка карт, сохранения данных аватара, обращения к статистике и проверки маршрутов, связанных с игровыми серверами. Эти сценарии соответствуют основным пользовательским действиям в платформе.

Отдельно проверялась устойчивость при частичных ошибках загрузки. Для сетевой игровой платформы важно, чтобы отсутствие второстепенной информации не приводило к полной остановке клиента. Поэтому данные профиля, друзей, каталога и карт должны загружаться последовательно и с обработкой ошибок.

Для проверки фактического состояния базы был получен JSON-отчет административной статистики. Он был сохранен в папку практики и использован при подготовке отчета. Такой подход делает результаты проверяемыми: преподаватель может видеть не только описание, но и реальные числовые показатели.

По итогам проверки серверный модуль был усилен: административная статистика стала включать размеры базы и файлового хранилища, а также количества записей по основным коллекциям. Это улучшает наблюдаемость проекта и делает сетевой модуль более пригодным для сопровождения.

| Проверка                    | Результат                                                              |
|-----------------------------|------------------------------------------------------------------------|
| Синтаксис Node.js API       | Файл server.js проходит проверку node --check                          |
| Проверка проекта Godot      | Проект загружается в headless-режиме без критических ошибок компиляции |
| Публичный health endpoint   | Сервер возвращает состояние API                                        |
| Административная статистика | Получены показатели базы данных и хранилища                            |
| Проверка структуры отчета   | Документ собран по шаблону примера УТП                                 |

Сведения, приведенные в таблице, используются в сетевом модуле как контрольные точки: по ним можно проверить, какие данные хранятся на сервере, какие операции выполняет клиент и какие результаты должны быть получены при демонстрации проекта.



Автономная некоммерческая профессиональная образовательная организация

**«МЕЖДУНАРОДНЫЙ ВОСТОЧНО-ЕВРОПЕЙСКИЙ КОЛЛЕДЖ»**

---

Пушкинская ул., д. 268, 426008, г. Ижевск. Тел.: (3412) 77-68-24. E-mail: mveu@mveu.ru, www.mveu.ru  
ИНН 1831200089. ОГРН 1201800020641

Такое представление удобно для защиты, поскольку показывает не только текстовое описание, но и связь между требованиями практики, архитектурой проекта и фактической реализацией в исходном коде.



## Заключение

В ходе учебно-технологической практики был разработан и оформлен сетевой модуль игровой платформы Bobux. Модуль объединяет клиент Godot, серверный API, базу данных PocketBase, файловое хранилище и игровой сервер. Он обеспечивает регистрацию, авторизацию, загрузку профиля, работу с пользовательским контентом, каталогом, инвентарем, друзьями и активными игровыми комнатами.

Разработанная архитектура соответствует принципу централизованного хранения данных. Клиент не является главным источником состояния, а обращается к серверу, который проверяет запросы и работает с базой. Это делает систему более надежной и позволяет развивать ее как полноценную игровую платформу.

В проекте выполнены требования, характерные для веб-сервиса с базой данных: реализованы учетные записи, авторизация, вывод информации из базы, загрузка файлов, создание и изменение записей, разграничение доступа и серверная проверка данных. При этом предметная область адаптирована под игровую платформу, а не под стандартный интернет-магазин.

Практическая ценность работы состоит в том, что сетевой модуль уже содержит реальные данные и может быть показан на защите. В базе имеются пользователи, карты, предметы, записи инвентаря и социальные связи. Административная статистика позволяет подтвердить состояние системы количественно.

В дальнейшем проект может быть расширен улучшенным интерфейсом администратора, более подробной системой ролей, очередями игровых серверов, журналом событий, расширенной модерацией пользовательского контента и аналитикой активности игроков.



## Список литературы

1. Документация Godot Engine 4. URL: <https://docs.godotengine.org/>
2. Документация PocketBase. URL: <https://pocketbase.io/docs/>
3. Документация Node.js. URL: <https://nodejs.org/>
4. Документация Express. URL: <https://expressjs.com/>
5. MDN Web Docs: HTTP, REST и работа с web API. URL: <https://developer.mozilla.org/>
6. Методические указания к выполнению примерного задания по учебно-технологической практике.
7. Материалы задания на учебно-технологическую практику, выданные преподавателем.
8. Исходный код проекта Bobux: клиент Godot, серверный API, сетевой модуль и инструменты развертывания.



## Приложение А. Основные файлы проекта

| Файл                                                | Назначение                                               |
|-----------------------------------------------------|----------------------------------------------------------|
| autoload/cloud_api.gd                               | Клиентский модуль HTTP-запросов к серверному API         |
| autoload/client_network.gd                          | Подключение клиента к игровому серверу                   |
| services/bobux_api/server.js                        | Основной серверный API, авторизация, данные и статистика |
| server/server_main.gd                               | Headless-игровой сервер и регистрация игровой комнаты    |
| tools/deploy_bobux_release.ps1                      | Сборка и развертывание релизов                           |
| практика/bobux_network_module_stats_2026-07-03.json | Снимок административной статистики базы данных           |

Данные файлы образуют основу сетевого модуля. Клиентская часть отвечает за запросы и подключение, серверный API управляет данными и безопасностью, а отдельный игровой сервер обеспечивает сетевые игровые сессии.



## Приложение Б. Сценарий демонстрации

Для защиты проекта рекомендуется демонстрировать не весь игровой проект целиком, а именно сетевой модуль как самостоятельный результат практики. Такой подход соответствует согласованной теме и позволяет показать преподавателю конкретную разработанную часть платформы.

Демонстрацию можно провести в следующем порядке:

1. Открыть клиент Vobux и показать форму входа, объяснив, что пользователь не получает доступ к профилю и данным без авторизации.
2. Выполнить вход в учетную запись и показать, что после авторизации клиент загружает профиль, список карт, элементы аватара и связанные пользовательские данные.
3. Открыть раздел карт или каталога и пояснить, что информация приходит из базы данных RocketBase через серверный API, а не хранится локально в клиенте.
4. Показать статистику базы данных из подготовленного JSON-файла или административного endpoint: количество пользователей, профилей, карт, предметов, записей инвентаря и размер хранилища.
5. Пояснить принцип работы игрового сервера: лобби получает сведения об активных комнатах, а игровой сервер отдельно отвечает за синхронизацию игроков в режиме.
6. Завершить демонстрацию схемой архитектуры и подчеркнуть, что сервер является главным источником данных, а клиент только отображает состояние и отправляет запросы.

Если преподаватель задаст вопрос о роли нейросетевых инструментов, корректная формулировка следующая: нейросетевые агенты использовались как вспомогательный инструмент анализа, документирования и ускорения разработки, однако сетевой модуль оформлен как инженерный результат проекта с проверяемой архитектурой, исходными файлами, серверной базой и фактическими метриками.